

PP1 - Evaluating Test Coverage Against Software Requirements – Formalism

April 2026

Contents

1	Introduction	3
1.1	The purpose of the formal base	3
1.2	The design principles of the formal base	3
2	The primitives	3
3	Entities and entity types	4
4	Valuations and traces	5
5	Expressions and predicates	6
5.1	Expressions	6
5.1.1	Constants	6
5.1.2	Arithmetic operators	6
5.1.3	Time functions	6
5.1.4	Entity attribute accessors	7
5.1.5	Trace functions	7
5.1.6	Set functions	8
5.2	Predicates	8
6	Operations	9
7	Temporal constructs	11
8	Requirements & Test cases	12
9	Design Decisions	13
10	Current Limitations	13
11	Future Extensions	13
12	Examples	16
12.1	Custom Counter	16
12.2	Req 01 – Arithmetic recurrence with condition	16
12.3	Req 02 – Store to non-volatile memory	17
12.4	Req 03 – Ordered read then store to CPU register	17
12.5	Req 04 – Exception vector table configuration	18
12.6	Req 05 – Exception handling procedure	18
12.7	Req 06 – Scheduling deadline	19
12.8	Req 07 – Mode enable transmission after initialisation	20

12.9 Req 08 – State transition on received mastership data	20
12.10Req 09 – Timed signal toggle with multi-condition guard	21
12.11Req 13 – Bounded count of large register values	22
12.12Req 10 – Repeated failure triggers emergency mode	23

1 Introduction

- **TODO:** [Value transformations, like signed, etc.]
- **TODO:** [Solve channel sync/async]

1.1 The purpose of the formal base

This document defines a logic-agnostic formal base for expressing software requirements and test cases. The goal is to enable mechanical detection of coverage gaps: given a set of requirements and a set of test cases, the checker determines which requirement conditions are not exercised by any test.

1.2 The design principles of the formal base

The formalism is designed around these properties.

- **extensibility** – adding new entity kinds or operations must require only extending the entity type set T_E , the values set V , the entity type modifiers function M_E , and the entity type operations function O_T .
- **matchability** – a formalised requirement and a formalised test case must be expressed using the same **predicates**, so that the coverage checker can determine which truth assignments to those predicates are exercised by the tests and which logical combinations remain uncovered.
- **ambient evaluation** – every expression is implicitly a function of a trace ρ and a current time Now . It is the surrounding temporal construct (e.g. *Always*, *Causes*, *CausesWithin*) that supplies Now by quantifying over time; expressions and operations never take Now as an explicit argument. This means the same expression can be reused inside different temporal constructs and its meaning changes solely by what the construct binds Now to.

2 The primitives

These primitive sets serve as the base that the rest of the formalism builds on. The primitive sets are the following:

- T_D, T_C – the *discrete time* and *continuous time* sets respectively.
- ID_E – the set of entity identifiers.
- V_{base} – the base values set: booleans, reals, time points, and intervals.
- V_{nonset}, V – defined in Section 3 once entities are available; V_{nonset} is the union of base values and entities; V additionally includes finite sets whose elements are drawn from V_{nonset} .
- $EventTypeLabel$ – the set of event type labels; defined below.

Definition 1 (Discrete time set). *The discrete time set T_D is totally ordered and countable, extended with a greatest element ∞ :*

$$T_D = \mathbb{N} \cup \{\infty\}$$

The ordering is the usual order on $\mathbb{N}$ with $\forall n \in \mathbb{N} : n < \infty$.

Definition 2 (Continuous time set). *The continuous time set T_C is totally ordered and uncountable, extended with a greatest element ∞ :*

$$T_C = \mathbb{R}_{\geq 0} \cup \{\infty\}$$

The ordering is the usual order on $\mathbb{R}_{\geq 0}$ with $\forall x \in \mathbb{R}_{\geq 0} : x < \infty$.

We say that T is an infinite totally ordered set; the time set can have *discrete* or *continuous* flavour; In the scope of a single requirement, T is equal either to T_D or T_C .

Definition 3 (Duration). A duration is a non-negative element of the time set, representing the length of a time span:

$$\Delta = T$$

Thus $\Delta = \mathbb{N}$ for discrete time and $\Delta = \mathbb{R}_{\geq 0}$ for continuous time.

Definition 4 (Interval). An interval is a tuple of a start time point, a duration, and two endpoint-openness flags:

$$\mathcal{I} = T \times \Delta \times \mathbb{B} \times \mathbb{B}$$

An interval $(t, d, \ell, r) \in \mathcal{I}$ represents a time span with start t , end $t + d$, left-open flag ℓ , and right-open flag r :

ℓ	r	represented span
false	false	$[t, t + d]$
true	false	$(t, t + d]$
false	true	$[t, t + d)$
true	true	$(t, t + d)$

Definition 5 (Entity identifier set). The entity identifier set ID_E is a finite nonempty set of unique abstract tokens called entity identifiers.

Definition 6 (Base values). The base values set V_{base} is:

$$V_{base} = \mathbb{B} \cup \mathbb{R} \cup T \cup \mathcal{I}$$

where $\mathbb{B} = \{\text{true}, \text{false}\}$ is the set of booleans, $\mathbb{R}$ is the set of real numbers, T is the time set, and $\mathcal{I}$ is the set of intervals.

Definition 7 (Event type label set). The event type label set $EventTypeLabel$ is a finite set of abstract symbols:

$$EventTypeLabel = \{ \text{generic}, \text{calculated}, \text{written}, \text{read}, \text{entered}, \text{transmitted}, \\ \text{received}, \text{inserted}, \text{removed}, \text{cleared} \}$$

3 Entities and entity types

An entity type is an item belonging to a set of entity types. There is a number of defined entity types and this set is open to extension.

Definition 8 (Entity type set).

$$T_E = \{ \text{Signal}, \text{Storage}, \text{Event}, \text{EventTrigger}, \text{Channel}, \text{State}, \text{Value}, \text{Abstract}, \text{Set} \}$$

Entity type modifiers are properties which can be attached to an entity of a particular type. A modifier is either a *flag*, whose presence alone carries meaning, or a *parameterised modifier*, which additionally carries a value in $ModVal$.

Definition 9 (Modifier value set). The modifier value set is:

$$ModVal = V_{base} \cup ID_E \cup EventTypeLabel$$

Each entity type is compatible with only a subset of the modifier keys. Therefore we define the *entity type modifiers function*.

Definition 10 (Entity type modifiers function). $M_E : T_E \rightarrow \mathcal{P}(ModKey)$ maps each entity type to its set of permitted modifier keys, where $ModKey$ is a finite nonempty set of modifier keys. The permitted modifier keys and their expected value types are:

<i>Entity type</i>	<i>Modifier key</i>	<i>Value type</i>
<i>Storage</i>	<i>address</i>	$\mathbb{N}$
<i>Event</i>	<i>target</i>	ID_E
<i>Event</i>	<i>type</i>	<i>EventTypeLabel</i>

Every **Event** entity is expected to carry both modifiers: user-declared events set *target* to the identifier of their **EventTrigger** and *type* to generic; predefined events set *target* to their parent entity and *type* to the appropriate event type.

Note: At most one **Event** entity with a given $(\text{target}, \text{type})$ pair should exist in E for a given requirement.

An entity is something that is referenced in a requirement and corresponding test cases. Formally, an entity is defined as a three-item tuple.

Definition 11 (Entity).

$$e = (id_e, t_e, \mu_e)$$

, where

- id_e is the unique identifier of the entity; $id_e \in ID_E$
- t_e is the type of the entity; $t_e \in T_E$
- μ_e is the modifier assignment function of the entity; $\mu_e : ModKey \rightarrow ModVal$, $\text{dom}(\mu_e) \subseteq M_E(t_e)$ ¹

No two entities in E share the same identifier: $\forall e, e' \in E : e \neq e' \Rightarrow id_e \neq id_{e'}$. We write E for the set of all entities. The set E is finite and nonempty.²

TODO: [Need to define Val_set]

Definition 12 (Non-set values). The non-set values V_{nonset} is the union of base values and entities:

$$V_{nonset} = V_{base} \cup E$$

Definition 13 (Values). The values set V is:

$$V = V_{nonset} \cup \mathcal{P}_{fin}(V_{nonset})$$

where $\mathcal{P}_{fin}(V_{nonset})$ is the set of all finite subsets of V_{nonset} . Set values are flat: they may contain only non-set values.

TODO: [We might need non-flat sets at some point. We should make a decision on that.]

4 Valuations and traces

Definition 14 (Valuation). A valuation is a partial function $\sigma : E \rightarrow V$ assigning a value to a subset of entities at each time point³; $\sigma(e)$ is undefined when the entity has no defined value at that time point. For entities of type **Event**, $\sigma(e)$ is always defined and $\sigma(e) \in \{\text{true}, \text{false}\}$, where $\sigma(e) = \text{true}$ iff the event occurs at that time point. For entities of type **Set**, $\sigma(e)$, when defined, is a finite subset of V_{nonset} , i.e. $\sigma(e) \in \mathcal{P}_{fin}(V_{nonset})$.

We write Σ for the infinite set of all valuations.

Definition 15 (Trace). A trace is a function $\rho : T \rightarrow \Sigma$. We write $\rho(t)(e)$ for the value of entity e at time t .

We write Σ^T for the set of all traces, i.e. the set of all functions from T to Σ .

4

¹ $\text{dom}(\mu_e)$ is a subset of the set of modifier keys compatible with the entity type t_e . It is not required that a value is defined for each modifier key compatible with t_e .

²The set E is finite in the scope of a single requirement.

³For instance a register entity can have its value undefined at time points preceeding a write to this register.

⁴This is standard function space notation: $\Sigma^T = \{\rho \mid \rho : T \rightarrow \Sigma\}$.

5 Expressions and predicates

All expression functions defined in this section evaluate to some value $v \in V$. We write **Expr** for the infinite set of all expressions.

5.1 Expressions

5.1.1 Constants

Definition 16 (Constant). *Any value $c \in V$ may appear as a constant expression.*

5.1.2 Arithmetic operators

Definition 17 (Arithmetic operators). *The standard arithmetic operators $+$, $-$, $\times$, $/$ are defined on numeric values ($\mathbb{R}$ and its subsets) in the usual sense and can be used to construct expressions.*

5.1.3 Time functions

The following functions depend only on time points or intervals; they do not depend on a trace.

Definition 18 (Current time). *Now $\in T$ returns the ambient time point t :*

$$\text{Now} = t$$

The following two functions are defined for **discrete time only** ($T = T_D = \mathbb{N}$).

Definition 19. *Prev $: T_D \rightarrow T_D$ returns the preceding time step:*

$$\text{Prev}(t) = t - 1$$

The function is partial: it is undefined at $t = 0$ since $t - 1 \notin T_D$.

Definition 20. *Next $: T_D \rightarrow T_D$ returns the following time step:*

$$\text{Next}(t) = t + 1$$

Definition 21. *RelTime $: T_D \times \mathbb{Z} \rightarrow T_D$ shifts a time point by n steps, where $n \in \mathbb{Z}$ may be positive or negative:*

$$\text{RelTime}(t, n) = t + n$$

The function is partial: it is undefined when $t + n < 0$ since $t + n \notin T_D$. Note that $\text{Prev}(t) = \text{RelTime}(t, -1)$ and $\text{Next}(t) = \text{RelTime}(t, 1)$.

The following functions are defined for both time flavours.

Definition 22. *Diff $: T \times T \rightarrow \Delta$ returns the duration between two time points:*

$$\text{Diff}(t_1, t_2) = |t_1 - t_2|$$

Definition 23 (Interval constructors). *Four constructors build intervals from a start and end time point, differing only in endpoint openness. All are partial: undefined when $t_1 > t_2$.*

$$\begin{aligned} \text{MkIntervalCC}(t_1, t_2) &= (t_1, t_2 - t_1, \text{false}, \text{false}) & [t_1, t_2] \\ \text{MkIntervalOC}(t_1, t_2) &= (t_1, t_2 - t_1, \text{true}, \text{false}) & (t_1, t_2] \\ \text{MkIntervalCO}(t_1, t_2) &= (t_1, t_2 - t_1, \text{false}, \text{true}) & [t_1, t_2) \\ \text{MkIntervalOO}(t_1, t_2) &= (t_1, t_2 - t_1, \text{true}, \text{true}) & (t_1, t_2) \end{aligned}$$

Definition 24 (Since-zero interval). *SinceZero $: T \rightarrow \mathcal{I}$ constructs an interval from time zero to a given time point t :*

$$\text{SinceZero}(t) = (0, t)$$

Definition 25 (Interval start). $Start : \mathcal{I} \rightarrow T$ returns the start time point of an interval:

$$Start((t, d, \ell, r)) = t$$

Definition 26 (Interval end). $End : \mathcal{I} \rightarrow T$ returns the end time point of an interval:

$$End((t, d, \ell, r)) = t + d$$

Definition 27 (Interval duration). $Duration : \mathcal{I} \rightarrow \Delta$ returns the duration of an interval:

$$Duration((t, d, \ell, r)) = d$$

Definition 28 (Interval endpoint openness). $IsLeftOpen, IsRightOpen : \mathcal{I} \rightarrow \mathbb{B}$ return the endpoint-openness flags:

$$IsLeftOpen((t, d, \ell, r)) = \ell \quad IsRightOpen((t, d, \ell, r)) = r$$

Definition 29 (Time point interval membership). $InInterval : T \times \mathcal{I} \rightarrow \mathbb{B}$ holds when time point s falls within interval i :

$$\begin{aligned} InInterval(s, i) \Leftrightarrow & (IsLeftOpen(i) \Rightarrow Start(i) < s) \\ & \wedge (\neg IsLeftOpen(i) \Rightarrow Start(i) \leq s) \\ & \wedge (IsRightOpen(i) \Rightarrow s < End(i)) \\ & \wedge (\neg IsRightOpen(i) \Rightarrow s \leq End(i)) \end{aligned}$$

5.1.4 Entity attribute accessors

These expressions return static properties of entities and do not depend on the trace or time.

Definition 30 (Address). For a *Storage* entity $e \in E$ carrying an address modifier, $Addr : E \rightarrow V$ returns the static address value:

$$Addr(e) = \mu_e(address)$$

5.1.5 Trace functions

The following functions additionally take a trace $\rho \in \Sigma^T$ and produce values that reflect the state of the system over time.

Definition 31. For a trace $\rho \in \Sigma^T$, the partial function $Val_\rho : E \times T \rightarrow V$ denotes the value of entity $e \in E$ at time $t \in T$:

$$Val_\rho(e, t) = \rho(t)(e)$$

The function is partial: $Val_\rho(e, t)$ is undefined when $e \notin \text{dom}(\rho(t))$.

Definition 32 (Value before time point). For a trace $\rho \in \Sigma^T$, the partial function $ValBefore_\rho : E \times T \rightarrow V$ is the left limit of the value of e at t :

$$ValBefore_\rho(e, t) = \lim_{t' \rightarrow t^-} Val_\rho(e, t')$$

The function is partial: it is undefined when the left limit does not exist, i.e. when no operation has fired on e before t .

Definition 33 (Event occurrence count). For a trace $\rho \in \Sigma^T$ and an entity $ev \in E$ of type *Event*, $EvtOccCount_\rho : E \times \mathcal{I} \rightarrow \mathbb{N}$ returns the number of occurrences of ev within an interval $i \in \mathcal{I}$:

$$EvtOccCount_\rho(ev, i) = |\{s \in T \mid InInterval(s, i) \wedge Val_\rho(ev, s) = true\}|$$

Definition 34 (Last occurrences function). For a trace $\rho \in \Sigma^T$, an entity $ev \in E$ of type **Event**, a time point $t \in T$, and $n \in \mathbb{N}^+$, let $t_1 \leq t_2 \leq \dots \leq t_n$ be the n most recent occurrence times of ev up to t , i.e. the n largest elements of $\{s \in T \mid s \leq t \wedge \text{Val}_\rho(ev, s) = \text{true}\}$ sorted in ascending order. Then $\text{LastOcc}_\rho : E \times T \times \mathbb{N}^+ \rightarrow \mathcal{I}$ is defined as:

$$\text{LastOcc}_\rho(ev, t, n) = \text{MkIntervalCC}(t_1, t_n)$$

The function is partial: it is undefined when fewer than n occurrences of ev exist up to t .

Note: The time of a single occurrence is obtained via $\text{LastOcc}_\rho(ev, t, 1)$, which returns an interval of duration 0. The occurrence time is then $\text{Start}(\text{LastOcc}_\rho(ev, t, 1))$.

Definition 35 (First occurrences function). For a trace $\rho \in \Sigma^T$, an entity $ev \in E$ of type **Event**, a time point $t \in T$, and $n \in \mathbb{N}^+$, let $t_1 \leq t_2 \leq \dots \leq t_n$ be the n earliest occurrence times of ev at or after t , i.e. the n smallest elements of $\{s \in T \mid s \geq t \wedge \text{Val}_\rho(ev, s) = \text{true}\}$ sorted in ascending order. Then $\text{FirstOcc}_\rho : E \times T \times \mathbb{N}^+ \rightarrow \mathcal{I}$ is defined as:

$$\text{FirstOcc}_\rho(ev, t, n) = \text{MkIntervalCC}(t_1, t_n)$$

The function is partial: it is undefined when fewer than n occurrences of ev exist at or after t .

Note: The time of the first occurrence is $\text{Start}(\text{FirstOcc}_\rho(ev, t, 1))$.

Definition 36 (Maximum value function). For a trace $\rho \in \Sigma^T$ and an entity $e \in E$ with an ordered value domain, $\text{MaxVal}_\rho : E \times \mathcal{I} \rightarrow V$ returns the maximum value of e over an interval $i \in \mathcal{I}$:

$$\text{MaxVal}_\rho(e, i) = \sup\{\text{Val}_\rho(e, s) \mid s \in T, \text{Start}(i) \leq s \leq \text{End}(i)\}$$

Definition 37 (Minimum value function). For a trace $\rho \in \Sigma^T$ and an entity $e \in E$ with an ordered value domain, $\text{MinVal}_\rho : E \times \mathcal{I} \rightarrow V$ returns the minimum value of e over an interval $i \in \mathcal{I}$:

$$\text{MinVal}_\rho(e, i) = \inf\{\text{Val}_\rho(e, s) \mid s \in T, \text{Start}(i) \leq s \leq \text{End}(i)\}$$

5.1.6 Set functions

These functions operate on set values and do not depend on a trace.

Definition 38 (Set size). $\text{Size} : \mathcal{P}_{\text{fin}}(V_{\text{nonset}}) \rightarrow \mathbb{N}$ returns the cardinality of a set:

$$\text{Size}(s) = |s|$$

Definition 39 (Set filter). For a set $s \in \mathcal{P}_{\text{fin}}(V_{\text{nonset}})$ and a predicate P parameterised by $x \in V_{\text{nonset}}$, $\text{Filter} : \mathcal{P}_{\text{fin}}(V_{\text{nonset}}) \times \mathbf{Pred} \rightarrow \mathcal{P}_{\text{fin}}(V_{\text{nonset}})$ returns the subset of elements satisfying P :

$$\text{Filter}(s, P) = \{x \in s \mid P(x)\}$$

5.2 Predicates

A predicate is a function that evaluates to a value in $\mathbb{B}$. We write $\mathbf{Pred}$ for the infinite set of all predicates.

Definition 40 (Boolean constants). $\top \in \mathbf{Pred}$ and $\perp \in \mathbf{Pred}$ are the always-true and always-false predicates respectively.

Definition 41 (Comparison predicate). For expressions $a, b \in V$ and a comparison operator $op \in \{=, \neq, <, \leq, >, \geq\}$:

$$\text{Cmp}(a, op, b) = a \text{ op } b$$

The operators $<, \leq, >, \geq$ require a, b to be from an ordered domain. Since $\Delta \subseteq \mathbb{R}$, durations are an ordered domain and duration comparison is a valid instance of Cmp .

Definition 42 (Set membership). For a value $v \in V_{\text{nonset}}$ and a set value $s \in \mathcal{P}_{\text{fin}}(V_{\text{nonset}})$:

$$\text{Contains}(v, s) \Leftrightarrow v \in s$$

Definition 43 (Negation). *For a predicate $P \in \mathbf{Pred}$:*

$$\text{Not}(P) = \neg P$$

Definition 44 (Conjunction and disjunction). *For a finite set of predicates $C \subseteq \mathbf{Pred}$:*

$$\text{AllOf}(C) = \bigwedge_{P \in C} P \quad \text{AnyOf}(C) = \bigvee_{P \in C} P$$

Definition 45 (Implication). *For predicates $A, B \in \mathbf{Pred}$:*

$$\text{Implies}(A, B) = A \Rightarrow B$$

where $\Rightarrow$ is the standard logical implication.

Definition 46 (Quantifiers). *For a finite set S and a predicate $P(x)$ parameterised by $x \in S$:*

$$\text{ForAll}(S, P) = \bigwedge_{x \in S} P(x) \quad \text{Exists}(S, P) = \bigvee_{x \in S} P(x)$$

Definition 47 (Event happening). *For an entity $e \in E$ with $t_e = \text{Event}$ and a time point $t \in T$:*

$$\text{Happening}(e, t) \Leftrightarrow \text{Val}_\rho(e, t) = \text{true}$$

Definition 48 (Event has happened). *For an entity $e \in E$ with $t_e = \text{Event}$ and a time point $t \in T$:*

$$\text{HasHappened}_\rho(e, t) \Leftrightarrow \exists t' \leq t : \text{Happening}(e, t')$$

The following predicates operate on intervals and durations. Since $\Delta \subseteq \mathbb{R}$, duration comparison is a special case of *Cmp* (Definition 41).

Definition 49 (Interval inclusion). *Interval i_1 includes i_2 iff i_2 is entirely contained within i_1 :*

$$\text{IntervalIncludes}(i_1, i_2) \Leftrightarrow \text{Start}(i_1) \leq \text{Start}(i_2) \wedge \text{End}(i_2) \leq \text{End}(i_1)$$

Definition 50 (Interval exclusion). *Intervals i_1 and i_2 are exclusive iff they are completely disjoint:*

$$\text{IntervalExcludes}(i_1, i_2) \Leftrightarrow \text{End}(i_1) < \text{Start}(i_2) \vee \text{End}(i_2) < \text{Start}(i_1)$$

Definition 51 (Start included). *The start of i_1 falls within i_2 :*

$$\text{StartOfFirstIntervalIn}(i_1, i_2) \Leftrightarrow \text{Start}(i_2) \leq \text{Start}(i_1) \leq \text{End}(i_2)$$

Definition 52 (End included). *The end of i_1 falls within i_2 :*

$$\text{EndOfFirstIntervalIn}(i_1, i_2) \Leftrightarrow \text{Start}(i_2) \leq \text{End}(i_1) \leq \text{End}(i_2)$$

6 Operations

An operation is an action that the system performs at a specific point in time. Each operation has a set of arguments and produces an *effect predicate* — a formal statement, parameterised by the firing time t , of what must hold in the trace as a result of the operation being performed. The operation is applied to some entity, which means that the argument list of every operation must include the entity the operation is applied to.

Definition 53 (Operation). *An operation is a function*

$$\text{op} : T \times E \times A_1 \times \cdots \times A_n \rightarrow \mathbf{Pred}$$

for some $n \geq 0$ and additional argument types $A_i \subseteq V \cup E$, where T is the time set and E is the mandatory target entity. The predicate $\text{op}(t, e, a_1, \dots, a_n)$ is the effect of op applied to entity e at time t — it describes the condition that the operation causes to hold. The trigger condition that invokes an operation is specified at the requirement level. We write O for the finite set of all operations.

Definition 54 (Entity type operations function).

$$O_T : T_E \rightarrow \mathcal{P}(O)$$

maps each entity type and set of modifiers to its applicable operations.

The concrete mapping, including argument types and effects, is as follows. In effect predicates, t_{next} denotes the next time *any* value-modifying operation fires on entity e after t , or ∞ if no such firing occurs:

$$t_{next} = \begin{cases} \min\{t' \in T \mid t' > t \wedge \text{a value-modifying } op' \in O_T(t_e) \text{ fires on } e \text{ at } t'\} & \text{if such } t' \text{ exists} \\ \infty & \text{otherwise} \end{cases}$$

, where t is the time of the current firing and value-modifying operations are those whose effect predicate contains a $Val_\rho(e, -)$ term (i.e. *calculate*, *write*, *set_state*, *transmit*, *insert*, *remove*, *clear*, and the destination side of *read* and *receive*).

In the effect column, $EvtPred(e, ek, t)$ is used as shorthand for:

$$\forall ev \in E : (t_{ev} = \text{Event} \wedge \mu_{ev}(\text{target}) = id_e \wedge \mu_{ev}(\text{type}) = ek) \Rightarrow Val_\rho(ev, t) = true$$

i.e. every **Event** entity declared with the matching *target* and *type* modifiers has its value set to *true* at t , which by definition causes $Happening(ev, t)$ to hold.

Operation	Notes	Effect predicates
$calculate(t, e, expr)$	$t \in T,$ $e \in E, t_e = \text{Signal},$ $expr \in \mathbf{Expr}$	$EvtPred(e, calculated, t)$ $\forall t' \in [t, t_{next}) : Val_\rho(e, t') = expr(\rho, t)$
$write(t, e, expr)$	$t \in T,$ $e \in E, t_e = \text{Storage},$ $expr \in \mathbf{Expr}$	$EvtPred(e, written, t)$ $\forall t' \in [t, t_{next}) : Val_\rho(e, t') = expr(\rho, t)$
$read(t, e_{src}, e_{dst})$	$t \in T,$ $e_{src} \in E, t_{e_{src}} = \text{Storage},$ $e_{dst} \in E, t_{e_{dst}} = \text{Signal}$	$EvtPred(e_{src}, read, t)$ $\forall t' \in [t, t_{next}) : Val_\rho(e_{dst}, t') = Val_\rho(e_{src}, t)$
$fire(t, e)$	$t \in T, e \in E, t_e = \text{EventTrigger}$	$EvtPred(e, generic, t)$
$set_state(t, e, s)$	$t \in T,$ $e \in E, t_e = \text{State},$ $s \in V$	$EvtPred(e, entered, t)$ $\forall t' \in [t, t_{next}) : Val_\rho(e, t') = s$
$transmit(t, e, expr)$	$t \in T,$ $e \in E, t_e = \text{Channel},$ $expr \in \mathbf{Expr}$	$EvtPred(e, transmitted, t)$ $\forall t' \in [t, t_{next}) : Val_\rho(e, t') = expr(\rho, t)$
$receive(t, e_{src}, e_{dst})$	$t \in T,$ $e_{src} \in E, t_{e_{src}} = \text{Channel},$ $e_{dst} \in E, t_{e_{dst}} = \text{Signal}$	$EvtPred(e_{src}, received, t)$ $Val_\rho(e_{dst}, t) = Val_\rho(e_{src}, t)$
$insert(t, e, v)$	$t \in T,$ $e \in E, t_e = \text{Set},$ $v \in V_{\text{nonset}}$	$EvtPred(e, inserted, t)$ $\forall t' \in [t, t_{next}) : Val_\rho(e, t') = ValBefore_\rho(e, t) \cup \{v\}$
$remove(t, e, v)$	$t \in T,$ $e \in E, t_e = \text{Set},$ $v \in V_{\text{nonset}}$	$EvtPred(e, removed, t)$ $\forall t' \in [t, t_{next}) : Val_\rho(e, t') = ValBefore_\rho(e, t) \setminus \{v\}$
$clear(t, e)$	$t \in T,$ $e \in E, t_e = \text{Set}$	$EvtPred(e, cleared, t)$ $\forall t' \in [t, t_{next}) : Val_\rho(e, t') = \emptyset$

7 Temporal constructs

So far, predicates ($\psi \in \mathbf{Pred}$) are point-in-time: they say whether something holds at a specific time t in a specific trace ρ . A *trace predicate* lifts this to the whole trace: it says whether something holds across the entire execution, not just at one moment. Formally, a trace predicate is a function $P : \Sigma^T \rightarrow \mathbb{B}$ — it takes a complete trace and returns true or false. A trace ρ *satisfies* a requirement R (written $\rho \models R$) iff R 's constraint, expressed as a trace predicate P , gives $P(\rho) = \text{true}$.

The temporal constructs below build trace predicates from point-in-time predicates $\psi \in \mathbf{Pred}$ by quantifying over time — expressing things like “ ψ must hold at all times” or “whenever ψ_{cond} holds, ψ_{effect} must eventually follow.”

Definition 55 (Trace predicate). A trace predicate is a function $P : \Sigma^T \rightarrow \mathbb{B}$. We write **TracePred** for the set of all trace predicates. A trace ρ satisfies a trace predicate P (written $\rho \models P$) iff $P(\rho) = \text{true}$.

Definition 56 (Temporal constructs).

$$\begin{aligned}
\text{Always}(\psi) &\Leftrightarrow \forall t \in T : \psi(\rho, t) \\
\text{Eventually}(\psi) &\Leftrightarrow \exists t \in T : \psi(\rho, t) \\
\text{Initial}(\psi) &\Leftrightarrow \psi(\rho, 0) \\
\text{Causes}(\psi_{\text{cond}}, \psi_{\text{effect}}) &\Leftrightarrow \forall t \in T : \psi_{\text{cond}}(\rho, t) \Rightarrow \exists t' \geq t : \psi_{\text{effect}}(\rho, t') \\
\text{CausesWithin}(\psi_{\text{cond}}, \psi_{\text{effect}}, d) &\Leftrightarrow \forall t \in T : \psi_{\text{cond}}(\rho, t) \Rightarrow \exists t' \in [t, t + d] : \psi_{\text{effect}}(\rho, t') \\
\text{Sequence}(\psi_1, \dots, \psi_n) &\Leftrightarrow \exists t_1 \leq t_2 \leq \dots \leq t_n \in T : \psi_i(\rho, t_i) \text{ for all } i \\
\text{Precedes}(\psi_1, \psi_2) &\Leftrightarrow \forall t_1 \in T : \psi_1(\rho, t_1) \Rightarrow \exists t_2 \leq t_1 : \psi_2(\rho, t_2) \\
\text{Excludes}(\psi_1, \psi_2) &\Leftrightarrow \forall t \in T : \neg(\psi_1(\rho, t) \wedge \psi_2(\rho, t)) \\
\text{Immediately}(\psi_1, \psi_2) &\Leftrightarrow \forall t_1 \in T : \psi_1(\rho, t_1) \Rightarrow \psi_2(\rho, \text{Next}(t_1))
\end{aligned}$$

where $\psi, \psi_{\text{cond}}, \psi_{\text{effect}}, \psi_1, \dots, \psi_n \in \mathbf{Pred}$ are point-in-time predicates and $d \in \Delta$ is a duration bound.

The constructs compose via standard logical operators on trace predicates: conjunction $P_1 \wedge P_2$, disjunction $P_1 \vee P_2$, negation $\neg P$, implication $P_1 \Rightarrow P_2$ (if P_1 holds for trace ρ then P_2 must also hold), and biconditional $P_1 \Leftrightarrow P_2$. These operate on whole traces, not at individual time points.

8 Requirements & Test cases

Both requirements and test cases operate over the same shared entity set E , which is what makes coverage checking possible.

A requirement is a formal constraint on the system's behaviour, expressed as a trace predicate.

Definition 57 (Requirement). A requirement is a triple $R = (\text{flavour}, \text{entities}, \text{constraint})$ where:

- *flavour* $\in \{\text{Discrete}, \text{Continuous}\}$ fixes the time set T for this requirement (T_D or T_C respectively), determining which expressions and temporal constructs are well-typed
- *entities* $\subseteq E$ is the set of entities participating in this requirement
- *constraint* $\in \mathbf{TracePred}$ is the formal constraint on the trace

A trace ρ satisfies R (written $\rho \models R$) iff $\text{constraint}(\rho) = \text{true}$.

A test case exercises a subset of the same entities to produce a concrete trace, which the coverage checker then analyses against the requirement's constraint.

Definition 58 (Test case). A test case is a pair $\text{TestCase} = (\text{setup}, \text{stimuli})$ where:

- *setup* is a valuation $\sigma_0 : E \rightarrow V$ establishing the initial state of the relevant entities
- *stimuli* $: T \rightarrow \mathcal{P}(O)$ maps each time point to the set of operations that fire at that time; it has finite support (non-empty for only finitely many t)

Together, *setup* and *stimuli* define a trace ρ . The coverage checker determines which predicates in a requirement's constraint are exercised by ρ .

9 Design Decisions

- Retaining the entity type as an explicit field is justified because it allows us to map each type to its own set of applicable modifiers and to its own set of applicable operations. **TODO:** [This might be outdated now.]

10 Current Limitations

- Modifiers need to be parameterized.
- **Partiality of time functions:** several functions are explicitly partial – $Prev$ is undefined at $t = 0$, $RelTime$ is undefined when $t + n < 0$, $MkInterval$ is undefined when $t_1 > t_2$, $ValBefore_\rho$ is undefined when no prior operation has fired, and Val_ρ is undefined when the entity has no value at the given time point. The formalism does not yet specify how partiality propagates when one of these appears inside a larger expression or predicate. For example, if $Now = 0$ then $Val_\rho(e, Prev(Now)) = 5$ contains an undefined sub-expression, and the truth value of the predicate is ambiguous. The options are:
 - *Well-formedness condition:* forbid such expressions syntactically, requiring the user to guard them explicitly (e.g. $Now > 0$).
 - *Three-valued / Kleene logic:* undefinedness propagates upward and yields a third truth value.
 - *Definedness guard:* any predicate containing an undefined sub-expression evaluates to *false*.
- **Open question – Channel/transmit semantics:** the transmitted value persists on the channel until the next *transmit*, so *receive* at any $t' \geq t$ reads the last transmitted value. It is unclear whether this is the right model for all channel types, or whether some channels should require the receiver to observe the value at the exact transmission time.

11 Future Extensions

- **Predicate snapshot comparison:** it may be useful to capture the *set of predicates that hold* at a given time point and compare that set with the one captured at a different time point. For example, one might want to assert that the set of predicates holding at time t_1 is equal to, a subset of, or has a non-empty intersection with the set holding at some later t_2 . A concrete use case is a reset requirement: “after a reset, the system must be in the same logical state as at initialisation”, where “logical state” is characterised by which predicates hold. This would require introducing a function $Snapshot_\rho : \mathcal{P}(\mathbf{Pred}) \times T \rightarrow \mathcal{P}(\mathbf{Pred})$ that returns the subset of a given predicate set that holds at a time point, and then allowing set operations ($=, \subseteq, \cap \neq \emptyset$) on the results.
- **Time-when expression:** add $LastTrue_\rho(\psi, t)$ returning the last $t' \leq t$ at which $\psi(\rho, t') = true$, and symmetrically $FirstTrue_\rho(\psi, t)$. Currently only $LastOcc_\rho$ and $FirstOcc_\rho$ cover this for **Event** entities specifically.
- **Time element for continuous requirements:** a requirement could optionally declare a *time element* $\varepsilon \in \mathbb{R}_{>0}$, representing the smallest meaningful time increment in that requirement (e.g. a sampling period or clock cycle). In the continuous-time flavour, $Prev$ and $Next$ are currently undefined because there is no canonical predecessor or successor of a real-valued time point. With a declared ε , they could be lifted to continuous time as $Prev(t) = t - \varepsilon$ and $Next(t) = t + \varepsilon$, making the full set of discrete-time expressions available to requirements that have a known sampling granularity.
- **Unified time model via time element:** the current formalism maintains two separate time flavours, $T_D = \mathbb{N}$ and $T_C = \mathbb{R}_{\geq 0}$, selected per requirement. These could be unified into a single model by making the time element ε (see the time element extension) a mandatory requirement parameter. The unified time set would then be $T = \{0, \varepsilon, 2\varepsilon, 3\varepsilon, \dots\}$, restricting time to multiples of ε and making $Prev(t) = t - \varepsilon$ and $Next(t) = t + \varepsilon$ total (except at $t = 0$). Under this model, discrete time is simply the special case $\varepsilon = 1$, and a “continuous” requirement with a known sampling period ε is expressed the

same way. Crucially, this is a stronger commitment than merely defining *Prev* and *Next* on an otherwise uncountable $\mathbb{R}_{\geq 0}$: it makes the time domain itself countable, so that quantifiers such as $\forall t \in T$ range over a countable set, which is important for coverage checking to remain tractable.

- **Open question – Synchronous channel model:** the current model is asynchronous – *transmit* and *receive* are independent operations that may occur at different times. An alternative synchronous model would have *transmit* directly cause *received* to fire at the same time t , making *receive* a consequence of *transmit* rather than a separate operation.
- **Typed sets**
- **Entity lookup by human-readable label.** A function $IdByLabel : \Gamma^* \rightarrow ID_E$ could resolve a short human-readable name to an entity identifier. The function is partial (undefined when no entity matches or the label is ambiguous). This is primarily a tooling concern: the formal handle is the entity identifier id_e ; how a user locates that identifier from natural language is outside the core formalism.
- **Required modifier keys.** The current M_E function specifies which modifier keys are *permitted* for each entity type, but all modifiers are optional. A future extension could introduce a companion function $M_E^{\text{req}} : T_E \rightarrow \mathcal{P}(\text{ModKey})$ specifying which modifier keys are *required*. For example, predefined-event **Event** entities (those used with *EvtPred*) should always have both *target* and *type* defined.
- **Aggregate expression recognition.** When a test case computes a result through a sequence of arithmetic operations (e.g. summing values and dividing by a count), the coverage checker must be able to recognise that the computation is equivalent to an aggregate such as average or median. Without this, a test exercising $avg(e, i)$ expressed via raw arithmetic would not be matched against a requirement that references the same aggregate by name. A future extension should define canonical aggregate expressions (*Avg*, *Median*, etc.) and a normalisation or equivalence procedure that maps common arithmetic patterns to them.
- **Triggered sequence construct.** *Sequence* is existential: it requires at least one complete chain of ordered events to occur somewhere in the trace. The universal reading — every occurrence of a triggering event must be followed by the full sequence — cannot be expressed with the current constructs. A *TriggeredSequence*($\psi_{\text{trigger}}, \psi_1, \dots, \psi_n$) construct would be needed, expanding to $\forall t \in T : \psi_{\text{trigger}}(\rho, t) \Rightarrow \exists t \leq t_1 \leq \dots \leq t_n : \psi_i(\rho, t_i)$ for all i .
- **Dynamic entity lifetimes.** The entity set E is fixed for the scope of a requirement, so the formalism cannot distinguish between an entity that does not exist at time t and one that merely has no value there — both reduce to $\sigma(e)$ being undefined. If entities with dynamic lifetimes are needed (e.g. a spawned process or an opened connection), an explicit existence predicate $Exists : E \times T \rightarrow \mathbb{B}$ should be introduced, with $Val_\rho(e, t)$ defined only when $Exists(e, t) = \text{true}$.
- **Enum types.** The base values set V_{base} currently contains only booleans, reals, time points, and intervals. Requirements that involve enumerated types (e.g. BACKUP/MASTER mastership modes, error codes, operating states) must represent enum values as untyped symbolic constants with no formal declaration or exhaustiveness guarantee. A future extension should introduce a named enum mechanism: a way to declare a finite set of symbolic constants (e.g. $Mastership = \{BACKUP, MASTER\}$) and extend V_{base} with their values, so that constraints can reference them by name and the coverage checker can enumerate their cases.
- **Event payload.** Currently an event entity carries only a boolean occurrence flag ($Val_\rho(ev, t) \in \mathbb{B}$); any data associated with the event must be expressed as a separate value check on the target entity (e.g. $Val_\rho(MODESET_TOPIC, Now) = \text{true}$ alongside $Happening(ev_transmitted, Now)$). This separates what logically belongs together and risks the value constraint being omitted in the requirement specification, silently leaving the payload not restricted to anything. A future extension should allow events to carry a typed payload, so that $Happening(ev, Now, v)$ asserts both that the event fired and that it carried value v .

- **User-defined modifier keys.** Currently, the set of permitted modifier keys for each entity type is fixed by M_E and can only be extended at the formalism level. A future extension could allow users to attach arbitrary key-value annotations to entities inline when writing a requirement – for example, to record a physical unit, an address, or a domain-specific tag – without requiring a change to M_E . One approach is to introduce a distinguished *user-defined* modifier namespace $ModKey_{user} \cap ModKey = \emptyset$ and extend M_E so that every entity type implicitly permits all keys from $ModKey_{user}$. The **Abstract** entity type already serves as a catch-all for entities that do not fit a predefined kind; user-defined modifiers would naturally complement it.

12 Examples

The examples in this section are sample requirements provided by the stakeholder rewritten into a formal form using an Claude Sonnet 4.6 with prompting guidance. That means the examples were not one-shotted. It seems so far that the model is not capable of one shotting these requirements.

12.1 Custom Counter

Requirement: After the system enters emergency mode, at each discrete time step the counter *counter_1* is incremented by 3.

Entity	t_e	μ_e
<i>enter_emergency</i>	EventTrigger	—
<i>ev_enter_emergency</i>	Event	$\{target \mapsto id_{enter_emergency}, type \mapsto generic\}$
<i>counter_1</i>	Signal	—

Constraint:

$$Always \left(HasHappened_{\rho}(ev_enter_emergency, Now) \Rightarrow Val_{\rho}(counter_1, Now) = Val_{\rho}(counter_1, Prev(Now)) + 3 \right)$$

The antecedent holds at time t iff *enter_emergency* has happened at least once up to t . The consequent requires *counter_1* to equal its previous value plus 3 at every such time step.

12.2 Req 01 – Arithmetic recurrence with condition

Requirement: The signal *EngagementNoSat_u* shall equal $1.2 \times EngagementNoSat_u(k-1)$ when *Switch_b* is true, and *Lon_u* otherwise. The initial value is 0.

Entity	t_e
<i>EngagementNoSat_u</i>	Signal
<i>Switch_b</i>	Signal
<i>Lon_u</i>	Signal

Flavour: Discrete.

Constraint:

$$\begin{aligned}
 &Initial(Val_{\rho}(EngagementNoSat_u, 0) = 0) \\
 \wedge \quad &Always \left(\begin{aligned}
 &(Val_{\rho}(Switch_b, Now) = true) \Rightarrow \\
 &\quad Val_{\rho}(EngagementNoSat_u, Now) = 1.2 \cdot Val_{\rho}(EngagementNoSat_u, Prev(Now)) \\
 &\wedge (Val_{\rho}(Switch_b, Now) \neq true) \Rightarrow \\
 &\quad Val_{\rho}(EngagementNoSat_u, Now) = Val_{\rho}(Lon_u, Now)
 \end{aligned} \right)
 \end{aligned}$$

12.3 Req 02 – Store to non-volatile memory

Requirement: The software shall store the maximum execution time measurement data in non-volatile memory at address *MEASUREMT_BLOCK*.

Entity	t_e	μ_e
<i>max_exec_time_data</i>	Abstract	—
<i>MEASUREMT_BLOCK</i>	Storage	$\{non_volatile \mapsto \top\}$
<i>reset</i>	EventTrigger	—
<i>ev_reset</i>	Event	$\{target \mapsto id_{reset}, type \mapsto generic\}$

Flavour: Discrete.

Constraint:

$$\begin{aligned}
& \text{Eventually} \left(Val_{\rho}(MEASUREMT_BLOCK, Now) = Val_{\rho}(max_exec_time_data, Now) \right) \\
& \wedge \text{Always} \left(\right. \\
& \quad \text{Happening}(ev_reset, Now) \Rightarrow \\
& \quad \left. Val_{\rho}(MEASUREMT_BLOCK, Next(Now)) = ValBefore_{\rho}(MEASUREMT_BLOCK, Now) \right)
\end{aligned}$$

12.4 Req 03 – Ordered read then store to CPU register

Requirement: The software must perform the following actions in the specified order: 1) Read the calibration constant value at the address 0xAA000018; 2) Store the calibration constant value in the DTSCON register.

Entity	t_e	μ_e
<i>calibration_const</i>	Storage	$\{address \mapsto 0xAA000018\}$
<i>DTSCON</i>	Storage	$\{register \mapsto \top\}$
<i>ev_read_cc</i>	Event	$\{target \mapsto id_{calibration_const}, type \mapsto read\}$
<i>ev_written_dtscon</i>	Event	$\{target \mapsto id_{DTSCON}, type \mapsto written\}$

Flavour: Discrete.

Constraint:

$$\begin{aligned}
& \text{Sequence} \left(\right. \\
& \quad \text{Happening}(ev_read_cc, Now), \\
& \quad \text{Happening}(ev_written_dtscon, Now) \\
& \quad \wedge Val_{\rho}(DTSCON, Now) \\
& \quad \left. = Val_{\rho}(calibration_const, Start(LastOcc_{\rho}(ev_read_cc, Now, 1))) \right)
\end{aligned}$$

12.5 Req 04 – Exception vector table configuration

Requirement: The software must configure IVORx registers with the addresses of Exception Vectors that compose the Exception Vector Table, according to the following table:

IVORx	Exception
IVOR0	Critical input
IVOR1	Machine check

Entity	t_e	μ_e
<i>IVOR0</i>	Storage	$\{register \mapsto \top\}$
<i>IVOR1</i>	Storage	$\{register \mapsto \top\}$
<i>CriticalInput</i>	Storage	—
<i>MachineCheck</i>	Storage	—

Flavour: Discrete.

Constraint:

$$Eventually\left(Val_{\rho}(IVOR0, Now) = Addr(CriticalInput) \wedge Val_{\rho}(IVOR1, Now) = Addr(MachineCheck)\right)$$

12.6 Req 05 – Exception handling procedure

Requirement: The software, when handling an exception, must execute the following steps in order: (1) copy register SRR0 to R5; (2) invoke the image exception handler; (3) if the handler returns, hold software execution.

Entity	t_e	μ_e
<i>SRR0</i>	Storage	$\{register \mapsto \top\}$
<i>R5</i>	Storage	$\{register \mapsto \top\}$
<i>ev_written_r5</i>	Event	$\{target \mapsto id_{R5}, type \mapsto written\}$
<i>exception</i>	EventTrigger	—
<i>ev_exception</i>	Event	$\{target \mapsto id_{exception}, type \mapsto generic\}$
<i>invoke_ieh</i>	EventTrigger	—
<i>ev_invoke_ieh</i>	Event	$\{target \mapsto id_{invoke_ieh}, type \mapsto generic\}$
<i>return_ieh</i>	EventTrigger	—
<i>ev_return_ieh</i>	Event	$\{target \mapsto id_{return_ieh}, type \mapsto generic\}$
<i>hold_exec</i>	EventTrigger	—
<i>ev_hold_exec</i>	Event	$\{target \mapsto id_{hold_exec}, type \mapsto generic\}$

Flavour: Discrete.

Constraint:

$$\begin{aligned}
& \text{Sequence} \Big(\\
& \quad \text{Happening}(ev_exception, Now), \\
& \quad \text{Happening}(ev_written_r5, Now) \\
& \quad \wedge Val_\rho(R5, Now) = Val_\rho(SRR0, Now), \\
& \quad \text{Happening}(ev_invoke_ieh, Now) \Big) \\
& \wedge \text{Causes} \Big(\text{Happening}(ev_return_ieh, Now), \text{Happening}(ev_hold_exec, Now) \Big)
\end{aligned}$$

Sequence is existential: it requires at least one complete chain to occur in the trace. The universal reading — every exception must trigger this sequence — would require a triggered-sequence construct not yet in the formalism. The *Causes* clause is conditional: it fires only if the handler returns, directly modelling the “if returns” branch.

12.7 Req 06 – Scheduling deadline

Requirement: The software must start executing the sequence of tasks allocated to the execution schedule in less than 2 seconds after processor power-up.

Entities:

Entity	t_e	μ_e
<i>processor_power_up</i>	EventTrigger	—
<i>ev_power_up</i>	Event	$\{target \mapsto id_{processor_power_up}, type \mapsto generic\}$
<i>invoke_task_seq</i>	EventTrigger	—
<i>ev_invoke_task_seq</i>	Event	$\{target \mapsto id_{invoke_task_seq}, type \mapsto generic\}$

Flavour: Continuous (time unit: seconds).

Constraint:

$$CausesWithin\left(Happening(ev_power_up, Now), Happening(ev_invoke_task_seq, Now), 2\right)$$

CausesWithin uses a closed interval $[t, t + 2]$, which admits exactly 2 s and slightly over-approximates the strict bound. An open-interval variant would be needed to express < 2 s exactly.

12.8 Req 07 – Mode enable transmission after initialisation

Requirement: After its initialisation, the software shall transmit the signal *Mode Enable* with value TRUE through DDS topic *MODESET_TOPIC*.

Entities:

Entity	t_e	μ_e
<i>MODESET_TOPIC</i>	Channel	—
<i>ev_transmitted_modeset</i>	Event	$\{target \mapsto id_{MODESET_TOPIC}, type \mapsto transmitted\}$
<i>initialization_end</i>	EventTrigger	—
<i>ev_init_end</i>	Event	$\{target \mapsto id_{initialization_end}, type \mapsto generic\}$

Flavour: Discrete.

Constraint:

$$Causes\left(\begin{array}{l} Happening(ev_init_end, Now), \\ Happening(ev_transmitted_modeset, Now) \\ \wedge Val_p(MODESET_TOPIC, Now) = true \end{array}\right)$$

The value check on *MODESET_TOPIC* is a separate predicate rather than part of the event itself, which is the event-payload limitation noted in the Future Extensions.

12.9 Req 08 – State transition on received mastership data

Requirement: The software shall operate as Backup when it receives a new data instance of *MastershipInfo* with parameter *Mastership_b* equal to BACKUP.

Entities:

Entity	t_e	μ_e
<i>MastershipInfo</i>	Channel	—
<i>ev_received_mastership</i>	Event	$\{target \mapsto id_{MastershipInfo}, type \mapsto received\}$
<i>Backup</i>	State	—
<i>ev_entered_backup</i>	Event	$\{target \mapsto id_{Backup}, type \mapsto entered\}$

Flavour: Discrete.

Constraint:

$$Causes \left(\begin{array}{l} \text{Happening}(ev_received_mastership, Now) \\ \wedge Val_\rho(MastershipInfo, Now) = BACKUP, \\ \text{Happening}(ev_entered_backup, Now) \end{array} \right)$$

BACKUP is an enum constant; enum types are not yet part of V_{base} and would require a future extension. The value check on *MastershipInfo* is a separate predicate rather than part of the event, which is the event-payload limitation noted in the Future Extensions.

TODO: [Need to add enum values]

12.10 Req 09 – Timed signal toggle with multi-condition guard

Requirement: The software must toggle *valid_range*, within 500ns of a write to *d*, if ALL the following conditions are satisfied:

- The difference between last Maximum Peak and Minimum Peak of *d* is higher than 655.
- It is the second write to *d* after the Minimum Peak detection.
- Signed value of *d* is higher than the signed value of last *d*.

Entities:

Entity	t_e	μ_e
<i>d</i>	Signal	—
<i>valid_range</i>	Signal	—
<i>ev_written_d</i>	Event	$\{target \mapsto id_d, type \mapsto written\}$
<i>ev_written_vr</i>	Event	$\{target \mapsto id_{valid_range}, type \mapsto written\}$
<i>min_peak_det</i>	EventTrigger	—
<i>ev_min_peak_det</i>	Event	$\{target \mapsto id_{min_peak_det}, type \mapsto generic\}$
<i>max_peak_det</i>	EventTrigger	—
<i>ev_max_peak_det</i>	Event	$\{target \mapsto id_{max_peak_det}, type \mapsto generic\}$

Flavour: Continuous (time unit: nanoseconds).

Let $t_{mpd} = \text{Start}(\text{LastOcc}_\rho(\text{ev_min_peak_det}, \text{Now}, 1))$, $t_{Mpd} = \text{Start}(\text{LastOcc}_\rho(\text{ev_max_peak_det}, \text{Now}, 1))$, and $t_{prevd} = \text{Start}(\text{LastOcc}_\rho(\text{ev_written_d}, \text{Now}, 2))$ abbreviate the times of the most recent minimum peak detection, maximum peak detection, and second-to-last write to d , all up to Now .

Constraint:

$$\begin{aligned} & \text{CausesWithin} \Big(\\ & \quad \text{Happening}(\text{ev_written_d}, \text{Now}) \\ & \quad \wedge \text{Val}_\rho(d, t_{Mpd}) - \text{Val}_\rho(d, t_{mpd}) > 655 \\ & \quad \wedge \text{EvtOccCount}_\rho(\text{ev_written_d}, \text{MkIntervalOC}(t_{mpd}, \text{Now})) = 2 \\ & \quad \wedge \text{Val}_\rho(d, \text{Now}) > \text{Val}_\rho(d, t_{prevd}), \\ & \quad \text{Happening}(\text{ev_written_vr}, \text{Now}) \\ & \quad \wedge \text{Val}_\rho(\text{valid_range}, \text{Now}) = \neg \text{ValBefore}_\rho(\text{valid_range}, \text{Now}), \\ & \quad 500 \Big) \end{aligned}$$

Condition 2: $\text{MkIntervalOC}(t_{mpd}, \text{Now})$ is open on the left, so a write coinciding exactly with the minimum peak detection at t_{mpd} is not counted, correctly capturing “after the Minimum Peak detection”.
Condition 3: “signed value of last d ” means the value of d at the previous write event, captured explicitly by $\text{Val}_\rho(d, t_{prevd})$. The signed integer interpretation has no counterpart in the formalism since d is treated as a real-valued signal. valid_range is assumed boolean; $\neg$ denotes boolean negation.

12.11 Req 13 – Bounded count of large register values

Requirement: The number of values larger than 10 in the four registers A, B, C, D when they are read at the same timestep is never greater than 2.

Entity	t_e	μ_e
A	Storage	$\{\text{register} \mapsto \top\}$
B	Storage	$\{\text{register} \mapsto \top\}$
C	Storage	$\{\text{register} \mapsto \top\}$
D	Storage	$\{\text{register} \mapsto \top\}$
ev_read_A	Event	$\{\text{target} \mapsto \text{id}_A, \text{type} \mapsto \text{read}\}$
ev_read_B	Event	$\{\text{target} \mapsto \text{id}_B, \text{type} \mapsto \text{read}\}$
ev_read_C	Event	$\{\text{target} \mapsto \text{id}_C, \text{type} \mapsto \text{read}\}$
ev_read_D	Event	$\{\text{target} \mapsto \text{id}_D, \text{type} \mapsto \text{read}\}$

Flavour: Discrete.

Constraint:

Let $\text{regs} = \{A, B, C, D\} \in \mathcal{P}_{\text{fin}}(V_{\text{nonset}})$ be the set of register entities. Both the antecedent and the consequent quantify over regs :

$$\begin{aligned}
& \text{Always} \left(\text{ForAll}(\text{regs}, \lambda e. \text{Happening}(e, \text{read}, \text{Now})) \right. \\
& \quad \left. \Rightarrow \text{Cmp} \left(\text{Size}(\text{Filter}(\text{regs}, \lambda e. \text{Val}_\rho(e, \text{Now}) > 10)), \leq, 2 \right) \right)
\end{aligned}$$

The antecedent holds at time t iff all four registers are read simultaneously. The consequent builds the set of registers whose value exceeds 10 and asserts its cardinality is at most 2.

12.12 Req 10 – Repeated failure triggers emergency mode

Requirement: The software shall enter the emergency mode if any of the following failures individually occurs twice in less than 10 seconds:

- (a) sensor failure
- (b) energy failure
- (c) communication failure.

Entities:

Entity	t_e	μ_e
<i>sensor_failure</i>	EventTrigger	—
<i>ev_sensor_fail</i>	Event	$\{target \mapsto id_{\text{sensor_failure}}, type \mapsto generic\}$
<i>energy_failure</i>	EventTrigger	—
<i>ev_energy_fail</i>	Event	$\{target \mapsto id_{\text{energy_failure}}, type \mapsto generic\}$
<i>comm_failure</i>	EventTrigger	—
<i>ev_comm_fail</i>	Event	$\{target \mapsto id_{\text{comm_failure}}, type \mapsto generic\}$
<i>emergency_mode</i>	State	—

Flavour: Continuous (time unit: seconds).

Constraint:

$$\begin{aligned}
& \text{Causes} \left(\left(\text{Happening}(ev_sensor_fail, \text{Now}) \right. \right. \\
& \quad \wedge \text{EvtOccCount}_\rho(ev_sensor_fail, \text{MkIntervalOO}(\text{Now} - 10, \text{Now})) \geq 1 \Big) \\
& \vee \left(\text{Happening}(ev_energy_fail, \text{Now}) \right. \\
& \quad \wedge \text{EvtOccCount}_\rho(ev_energy_fail, \text{MkIntervalOO}(\text{Now} - 10, \text{Now})) \geq 1 \Big) \\
& \vee \left(\text{Happening}(ev_comm_fail, \text{Now}) \right. \\
& \quad \wedge \text{EvtOccCount}_\rho(ev_comm_fail, \text{MkIntervalOO}(\text{Now} - 10, \text{Now})) \geq 1 \Big), \\
& \quad \left. \text{Val}_\rho(emergency_mode, \text{Now}) = true \right)
\end{aligned}$$

“*Individually*”: the three failure types are tracked separately; ψ is instantiated independently for each event entity, so a sensor failure and an energy failure occurring within 10 seconds of each other do not jointly

trigger the condition. “*Twice in less than 10 seconds*”: $MkIntervalOO(Now - 10, Now)$ is open on both ends: the left endpoint excludes a gap of exactly 10 seconds (“less than”), and the right endpoint excludes Now itself so that only the previous occurrence is counted alongside the current *Happening*. *Edge case*: when $Now < 10$, the expression $Now - 10$ is negative and outside $T = \mathbb{R}_{\geq 0}$; in practice no event can occur before time 0, so the count is still correct, but a well-formed guard $Now \geq 10$ would be needed to make the interval argument a valid element of T .